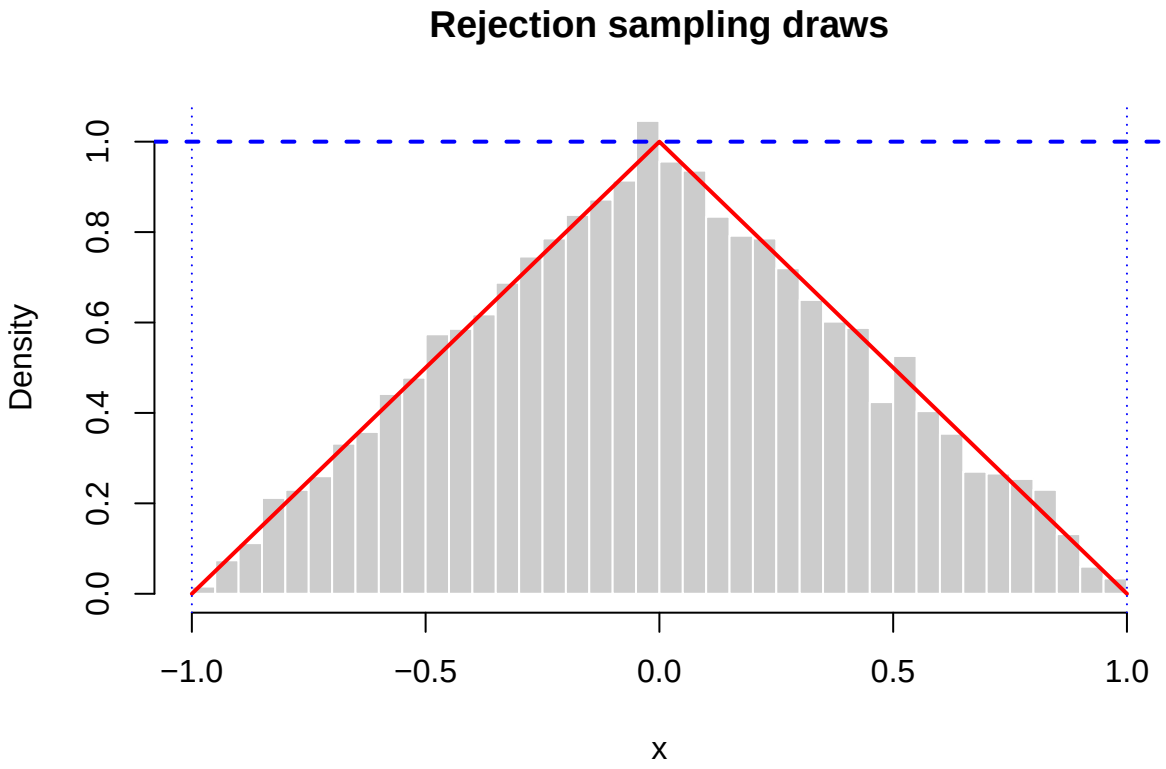
Lab 3

Aron Enström (aroen488), Sergey Volchkov (servo519)

2026-02-05

Q1:

1.1. Rejection sampling




```
## Rejection rate: 0.5006741
```

1.2. Composition sampling

Since X is defined as the mixture of Y and $-Y$, the composite density is just the mixture of their densities.

Given $Y \sim f_Y(y) = 2 - 2y$ on $[0, 1]$,

- Density of X on $[0, 1]$ (coming from Y) is

$$f_X(x) = \tfrac{1}{2}(2 - 2x) = 1 - x, \quad 0 \le x \le 1.$$

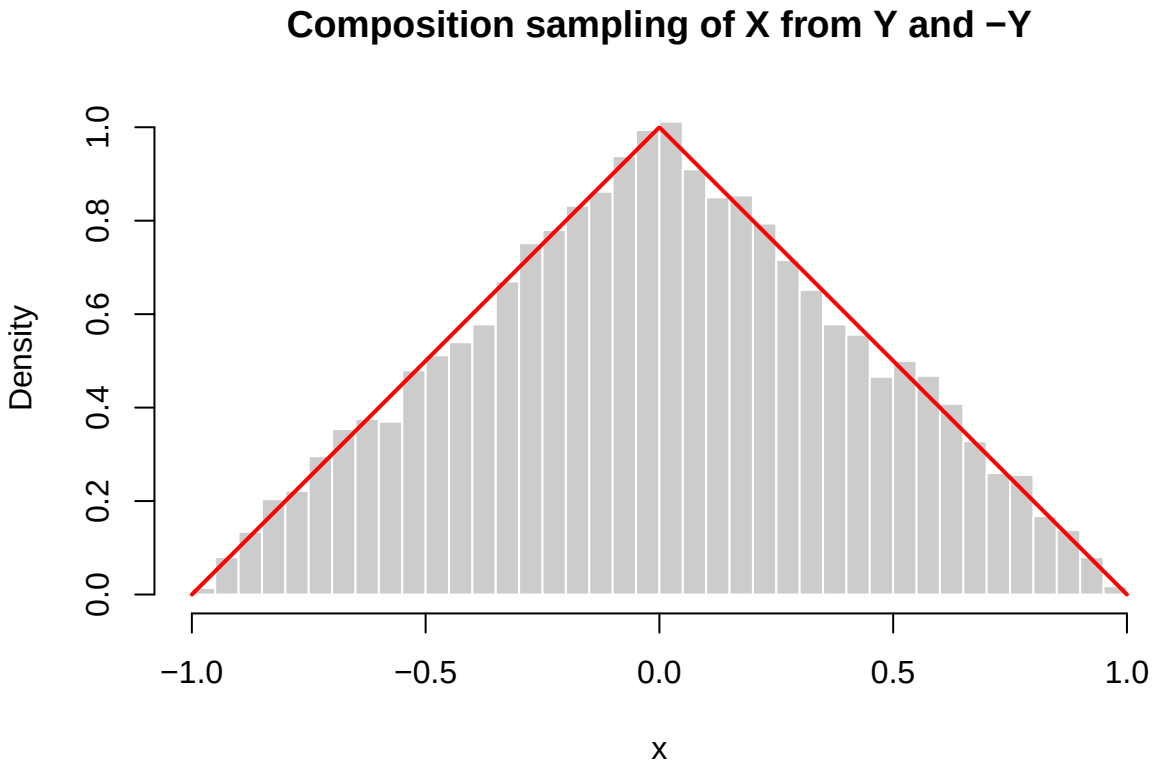
- Density of X on $[-1, 0]$ (coming from $-Y$) is

$$f_X(x) = \tfrac{1}{2}(2 - 2(-x)) = 1 + x, \quad -1 \le x \le 0.$$

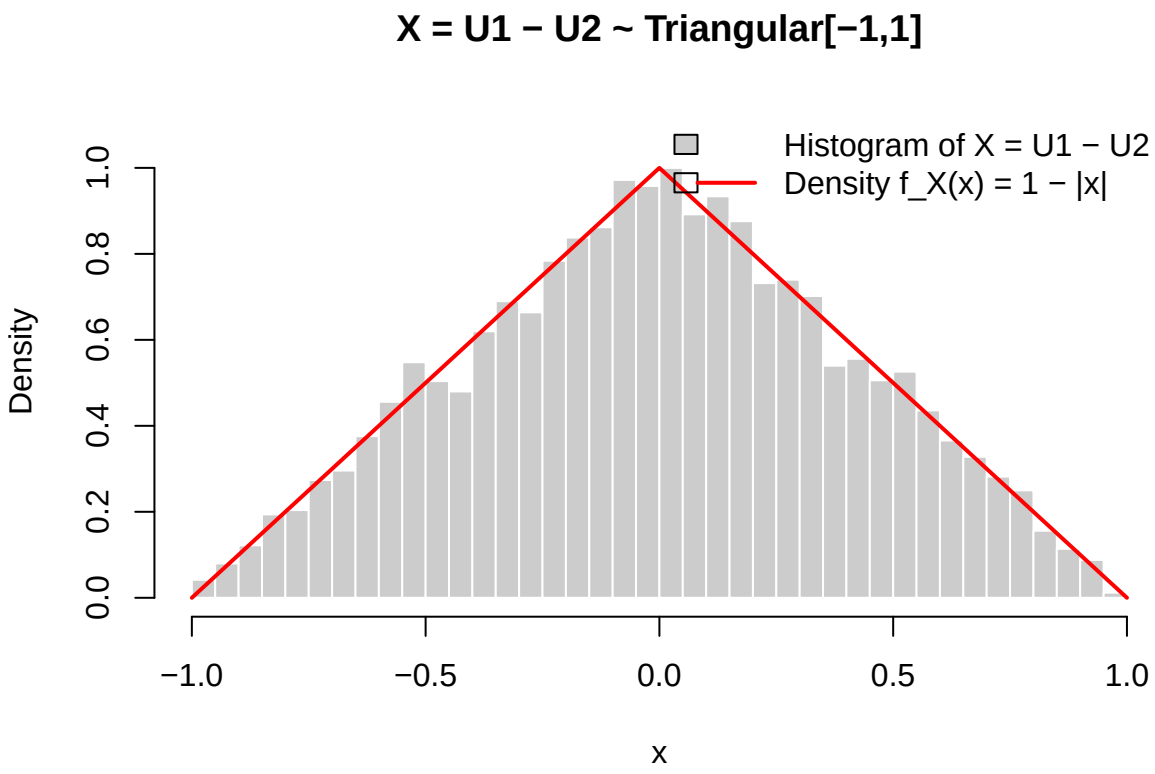
So overall

$$f_X(x) = \begin{cases} 1 + x, & -1 \le x \le 0, \\ 1 - x, & 0 \le x \le 1, \\ 0, & \text{otherwise.} \end{cases}$$

(same function as in 1.1)



1.3. Using a composition of two independent uniform variables



1.4. Comparing the methods

Lines of meaningful R code

Method	Core generator code (lines)
Rejection sampling	12 lines (loop with 2 `runif`, `if`, counters)
Composition (Y, -Y)	6 lines (2 generators + mixture)
U1 - U2	2 lines (`u1 - u2`)

Computational cost per sample

Method	Unif calls	Other ops per accepted sample	Expected total ops
Rejection	2	1 comparison + 50% loop overhead	~4 Unifs + 2 comparisons
Composition	1 (via Y generator)	1 `sqrt`, 1 conditional	1 Unif + 1 sqrt + 1 branch
U1 - U2	2	1 subtraction	2 Unifs + 1 sub

Expected running time judgment

1. U1 - U2 wins: No loops, no expensive `sqrt()`, no conditionals, no wasted proposals. Pure vectorized arithmetic on 10000 draws takes ~0.001s.

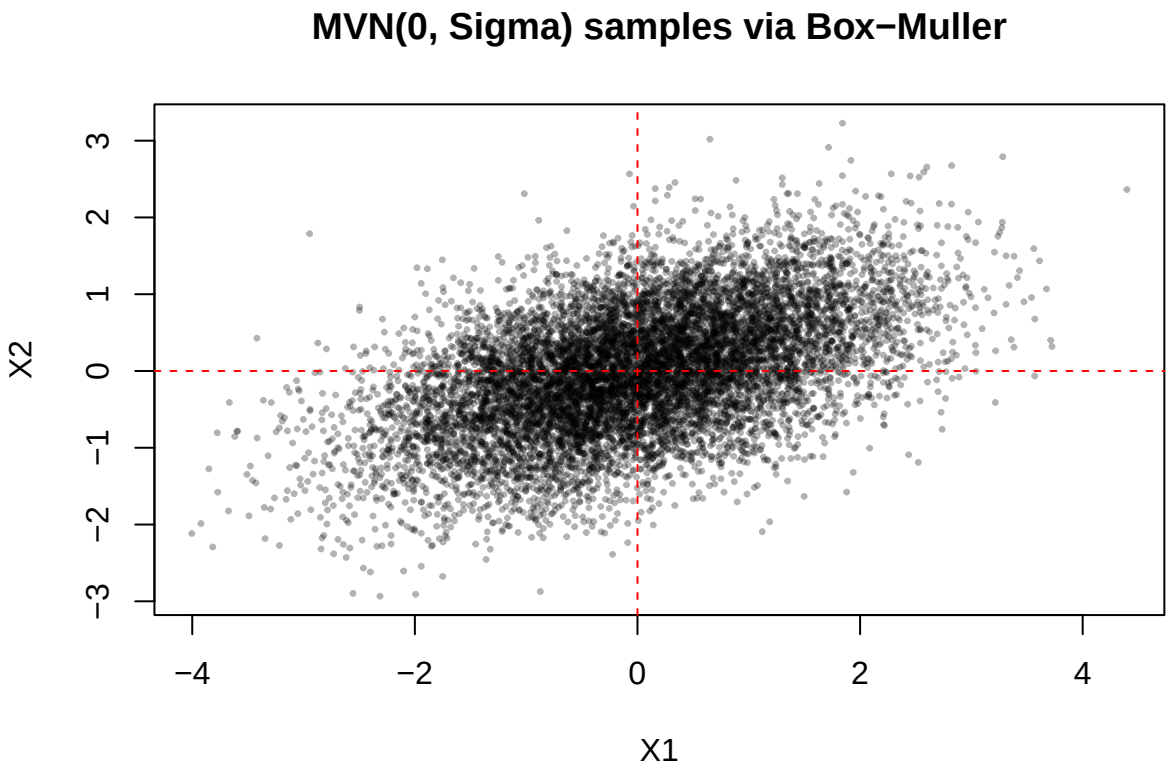
2. Rejection wastes 50%: The `while` loop doubles uniform generation (2 vs 1 needed), plus branch prediction misses and counter overhead. ~3-5× slower than U1-U2
3. Composition decent but hurt by sqrt: The `1-sqrt(1-u)` is ~10× slower than plain `runif` per your Y generator. Still beats rejection but loses to simple differences.

Preference U1 - U2. It's mathematically elegant (difference of uniforms = triangular), shortest code, fastest runtime, and perfectly vectorized. For millions of samples, the difference becomes dramatic; even for 10k it's noticeably snappier. Rejection sampling only shines when you lack a direct method.

Q2:

2.1. Generating samples with Box-Muller method

```
## Elapsed time in seconds: 0
```



2.2. Generating samples with mvtnorm

We chose `mvtnorm::rmvnorm` with Cholesky method because:

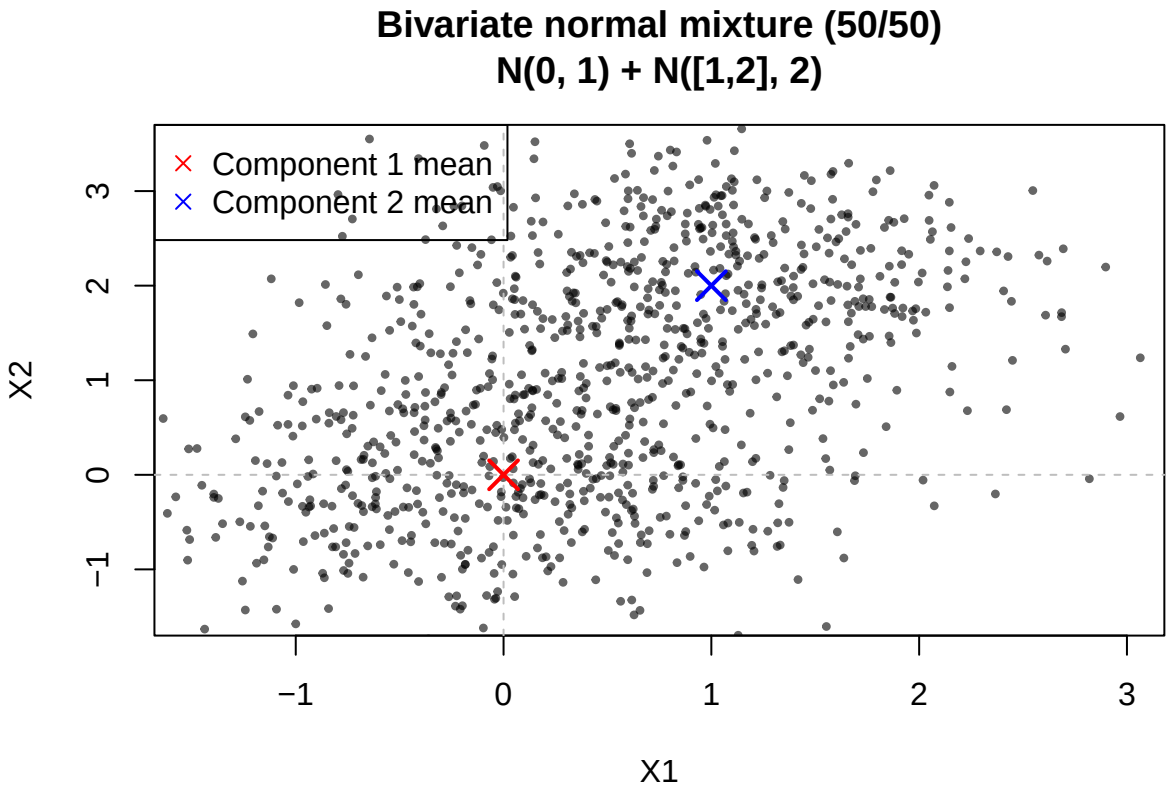
- It's production-optimized C/Fortran code, faster than pure R rnorm + manual Cholesky for large $n = 10^7$.
- Explicit method = "chol" matches our Box-Muller approach exactly (avoids slower SVD/eigen).
- Handles matrix arithmetic internally with BLAS/LAPACK acceleration.

```
## [1] "Box-Muller: 2.410 sec"
```

```
## [1] "mvtnorm::rmvnorm: 1.720 sec"
```

```
## [1] "mvtnorm / Box-Muller speedup: 1.4x"
```

2.3. Bivariate normal mixture



The plot is looking very satisfactory.

One can clearly see:

- Two distinct clusters: one near $(0, 0)$ (red), one near $(1, 2)$ (blue).
- Roughly equal sizes (~500 each), matching the 50/50 mixture.
- Circular/spherical spreads: variances 0.6 and 0.5, no correlation (diagonal Σ).
- Clean separation due to distant means relative to small std devs ($\sigma \approx 0.7, 0.71$).

With $n = 1000$, sampling variability is low enough to clearly reveal the mixture structure. Perfect demonstration of a bivariate Gaussian mixture.

Appendix

```r
#########
# 1.1
#########
f <- function(x) {
  ifelse(x < -1 | x > 1, 0,
         ifelse(x <= 0, x + 1, 1 - x))
}

rtri_reject <- function(n) {
  out <- numeric(n)
```

```r
  ntry <- 0L; i <- 1L
  while (i <= n) {
    x <- runif(1, -1, 1)
    u <- runif(1)
    ntry <- ntry + 1L
    if (u <= f(x)) { out[i] <- x; i <- i + 1L }
  }
  list(x = out, ntry = ntry, accept_rate = n/ntry, waste = 1 - n/ntry)
}

set.seed(1)
res <- rtri_reject(10000)

h <- hist(
  res$x, breaks = 60, freq = FALSE,
  col = "grey80", border = "white",
  main = "Rejection sampling draws", xlab = "x",
  xlim = c(-1, 1), plot = FALSE
)

hist(res$x, breaks = 60, freq = FALSE, col = "grey80", border = "white",
     main = "Rejection sampling draws", xlab = "x", xlim = c(-1, 1))

curve(f(x), from = -1, to = 1, add = TRUE, lwd = 2, col = "red")

## envelope e(x) = 1_{[-1,1]}(x)
abline(h = 1, col = "blue", lwd = 2, lty = 2)
abline(v = c(-1, 1), col = "blue", lwd = 1, lty = 3)

cat("Rejection rate:", res$waste)
#########
# 1.2
#########
set.seed(1)

## generator for Y ~ with density f_Y(y) = 2 - 2y, 0 <= y <= 1
rY <- function(n) {
  u <- runif(n)
  1 - sqrt(1 - u)
}

## composition sampler for X from Y and -Y
rX <- function(n) {
  y  <- rY(n)                  # base draws
  s  <- rbinom(n, size = 1, prob = 0.5)  # 0 or 1 with prob 1/2
  x  <- ifelse(s == 1,  y, -y)
  x
}

## generate 10000 draws
x <- rX(10000)

## histogram
```

```r
hist(x,
     breaks = 60,
     freq   = FALSE,
     col    = "grey80",
     border = "white",
     main   = "Composition sampling of X from Y and -Y",
     xlab   = "x",
     xlim   = c(-1, 1))

## composite density of X
fX <- function(x) {
  ifelse(x < -1 | x > 1, 0,
         ifelse(x <= 0, 1 + x, 1 - x))
}

## overlay theoretical composite density
xx <- seq(-1, 1, length.out = 1000)
lines(xx, fX(xx), col = "red", lwd = 2)
#########
# 1.3
#########
set.seed(1)

## Generator for X using U1 - U2
rX_udiff <- function(n) {
  u1 <- runif(n)
  u2 <- runif(n)
  u1 - u2
}

## Generate 10000 draws
x <- rX_udiff(10000)

## Histogram on density scale
hist(x,
     breaks = 60,
     freq = FALSE,
     col = "grey80",
     border = "white",
     main = "X = U1 - U2 ~ Triangular[-1,1]",
     xlab = "x",
     xlim = c(-1, 1),
     ylim = c(0, 1.1))

## Theoretical triangular density f_X(x) = 1 - |x| on [-1,1]
fX <- function(x) ifelse(abs(x) <= 1, 1 - abs(x), 0)
curve(fX, from = -1, to = 1, add = TRUE, lwd = 2, col = "red")

legend("topright",
       legend = c("Histogram of X = U1 - U2", "Density f_X(x) = 1 - |x|"),
       fill = c("grey80", NA),
       col = c(NA, "red"),
       lwd = c(NA, 2),
```

```
       bty = "n")
#########
#2.1
#########
set.seed(1)

## Box-Muller method for 2D multivariate normal
## mean c(0,0), covariance matrix [[1, 0.6], [0.6, 1]]
## generates N(0,1) iid standard normals Z1, Z2, then transforms

boxmuller_mvnorm <- function(n) {
  u1 <- runif(n)           # U1, U2 ~ Unif[0,1] iid
  u2 <- runif(n)

  r  <- sqrt(-2 * log(u1))
  theta <- 2 * pi * u2     # 2 pi U2

  z1 <- r * cos(theta)     # Z1, Z2 ~ N(0,1) iid
  z2 <- r * sin(theta)

  z <- cbind(z1, z2)

  ## Cholesky decomposition Sigma = L L^T, with L lower triangular
  sigma <- matrix(c(1, 0.6, 0.6, 1), 2, 2)
  L <- chol(sigma)

  ## X = mu + L Z
  mu <- c(0, 0)
  x <- t(L %*% t(z)) + mu  # n x 2 matrix

  x
}

n <- 10000
x <- boxmuller_mvnorm(n)

## Time measurement
t0 <- proc.time()
x <- boxmuller_mvnorm(n)
t1 <- proc.time()
cat("Elapsed time in seconds:", t1[3] - t0[3])

plot(x, pch = 20, cex = 0.5, col = rgb(0,0,0,0.3),
     main = "MVN(0, Sigma) samples via Box-Muller",
     xlab = "X1", ylab = "X2")
abline(h=0, v=0, col="red", lty=2)
#########
#2.2
#########
set.seed(1)
library(mvtnorm)

set.seed(1)
```

```r
n <- 1e7    # 10 million vectors
sigma <- matrix(c(1, 0.6, 0.6, 1), 2, 2)
mu <- c(0, 0)

## a) Box-Muller baseline (from previous)
t0 <- proc.time()
x_bm <- boxmuller_mvnorm(n)
time_bm <- (proc.time() - t0)[3]
print(sprintf("Box-Muller: %.3f sec", time_bm))

## b) mvtnorm
t1 <- proc.time()
x_mvtnorm <- rmvnorm(n, mean = mu, sigma = sigma, method = "chol")
time_mvtnorm <- (proc.time() - t1)[3]
print(sprintf("mvtnorm::rmvnorm: %.3f sec", time_mvtnorm))

## Comparison
print(sprintf("mvtnorm / Box-Muller speedup: %.1fx", time_bm / time_mvtnorm))
#########
#2.3
#########
set.seed(1)

n <- 1000

## Two component MVN parameters
mu1 <- c(0, 0)
sigma1 <- matrix(c(0.6, 0, 0, 0.6), 2, 2)

mu2 <- c(1, 2)
sigma2 <- matrix(c(0.5, 0, 0, 0.5), 2, 2)

## Generate mixture: 50/50 with prob 0.5 each
component <- rbinom(n, 1, 0.5)
x <- rbind(
  rmvnorm(sum(component == 0), mean = mu1, sigma = sigma1),
  rmvnorm(sum(component == 1), mean = mu2, sigma = sigma2)
)[sample(n), ]  # shuffle to mix

## Plot 1000 samples
plot(x, pch = 20, cex = 0.7, col = rgb(0,0,0,0.6),
     main = "Bivariate normal mixture (50/50)\nN(0, 1) + N([1,2], 2)",
     xlab = "X1", ylab = "X2",
     xlim = c(-1.5, 3), ylim = c(-1.5, 3.5))
abline(h = 0, v = 0, col = "grey", lty = 2)

## Theoretical component means (for reference)
points(mu1[1], mu1[2], pch = 4, cex = 2, col = "red", lwd = 2)
points(mu2[1], mu2[2], pch = 4, cex = 2, col = "blue", lwd = 2)
legend("topleft",
       legend = c("Component 1 mean", "Component 2 mean"),
       pch = 4, col = c("red", "blue"))
```